

Лабораторная работа №3.

JWT-аутентификация

Тема: Создание access JWT, Bearer authentication и защита Express endpoints

Цель: Реализовать выдачу JWT после успешного login, передачу access token через Authorization: Bearer и middleware requireAuth для защиты REST endpoints.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте работу с проектом express-shop. В проекте уже должны работать регистрация, login, passwordService.js, Argon2 и Prisma User.

Граница работы

В этой лабораторной реализуется короткоживущий access JWT. Refresh token и хранение JWT во frontend будут изучаться отдельно.

1. Установите jsonwebtoken

```
PowerShell  
  
npm install jsonwebtoken
```

2. Создайте JWT secret

Для HS256 нужен криптографически случайный секрет не короче 256 бит. Не используйте обычный пароль или короткую фразу.

```
PowerShell  
  
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Скопируйте полученную строку в .env. Не публикуйте реальное значение в Git.

```
.env  
  
JWT_SECRET=<your_random_secret>  
JWT_EXPIRES_IN=15m  
JWT_ISSUER=express-shop  
JWT_AUDIENCE=express-shop-api
```

Секрет только на сервере

JWT_SECRET нельзя помещать во frontend JavaScript, отправлять клиенту или сохранять в JWT payload.

3. Создайте tokenService

```
services/tokenService.js

import jwt from "jsonwebtoken";

export function createAccessToken(user) {
  return jwt.sign(
    {
      role: user.role
    },
    process.env.JWT_SECRET,
    {
      algorithm: "HS256",
      subject: String(user.id),
      issuer: process.env.JWT_ISSUER,
      audience: process.env.JWT_AUDIENCE,
      expiresIn: process.env.JWT_EXPIRES_IN
    }
  );
}

export function verifyAccessToken(token) {
  return jwt.verify(
    token,
    process.env.JWT_SECRET,
    {
      algorithms: ["HS256"],
      issuer: process.env.JWT_ISSUER,
      audience: process.env.JWT_AUDIENCE
    }
  );
}
```

4. Измените login Controller

```
controllers/authController.js

import { authenticateUser } from "../services/authService.js";
import { createAccessToken } from "../services/tokenService.js";

export async function login(req, res, next) {
  try {
    const { email, password } = req.body;
    const user = await authenticateUser(email, password);

    if (!user) {
      return res.status(401).json({
        error: "Invalid email or password"
      });
    }

    const accessToken = createAccessToken(user);

    res.json({
      user,
      accessToken
    });
  } catch (error) {
    next(error);
  }
}
```

```
}
```

JWT создаётся только после успешной проверки email и password. В token не помещается passwordHash.

5. Проверьте login

```
POST /api/auth/login
```

```
POST /api/auth/login
Content-Type: application/json
```

```
{
  "email": "alex@example.com",
  "password": "ExamplePassword123!"
}
```

Пример ответа

```
{
  "user": {
    "id": 1,
    "name": "Alex",
    "email": "alex@example.com",
    "role": "user"
  },
  "accessToken": "eyJ..."
}
```

Скопируйте accessToken. Он понадобится для проверки protected endpoint.

6. Изучите JWT

JWT состоит из трёх частей, разделённых точками: Header, Payload и Signature. Header и Payload можно декодировать без секрета, поэтому JWT payload не предназначен для хранения секретных данных.

- В Header должен быть указан алгоритм HS256.
- В Payload должен присутствовать sub с идентификатором User.
- В Payload должен присутствовать iat.
- В Payload должен присутствовать exp.
- Также должны присутствовать значения issuer и audience, заданные приложением.

7. Создайте функцию извлечения Bearer token

```
middleware/authMiddleware.js
```

```
import { verifyAccessToken } from "../services/tokenService.js";

function getBearerToken(req) {
  const header = req.headers.authorization;

  if (!header) {
    return null;
  }
}
```

```

const [scheme, token] = header.split(" ");

if (scheme !== "Bearer" || !token) {
  return null;
}

return token;
}

```

Ожидаемый формат заголовка: `Authorization: Bearer <token>`.

8. Реализуйте requireAuth

```

middleware/authMiddleware.js

export function requireAuth(req, res, next) {
  const token = getBearerToken(req);

  if (!token) {
    return res.status(401).json({
      error: "Authentication required"
    });
  }

  try {
    const payload = verifyAccessToken(token);

    req.user = {
      id: Number(payload.sub),
      role: payload.role
    };

    next();
  } catch {
    return res.status(401).json({
      error: "Invalid or expired token"
    });
  }
}

```

verify вместо decode

Protected endpoint должен доверять claims только после `jwt.verify()`. Простого декодирования JWT недостаточно.

9. Реализуйте GET /api/auth/me

```

controllers/authController.js

import { prisma } from "../../config/prisma.js";

export async function getCurrentUser(req, res, next) {
  try {
    const user = await prisma.user.findUnique({
      where: {
        id: req.user.id
      }
    });
  }
}

```

```

    },
    select: {
      id: true,
      name: true,
      email: true,
      role: true
    }
  });

  if (!user) {
    return res.status(401).json({
      error: "User not found"
    });
  }

  res.json({ user });
} catch (error) {
  next(error);
}
}

```

Для этой лабораторной допустимо получить User в Controller. Если в вашем проекте используется отдельный User Service, перенесите Prisma-запрос туда.

10. Защитите маршрут

```

routes/authRoutes.js

import { Router } from "express";
import {
  login,
  register,
  getCurrentUser
} from "../controllers/authController.js";
import { requireAuth } from "../middleware/authMiddleware.js";

const router = Router();

router.post("/register", register);
router.post("/login", login);
router.get("/me", requireAuth, getCurrentUser);

export default router;

```

11. Проверьте запрос без token

```
GET /api/auth/me
```

Ожидаемый ответ

```

{
  "error": "Authentication required"
}

```

Ожидаемый HTTP status — 401.

12. Проверьте запрос с Bearer token

```
GET /api/auth/me
```

```
GET /api/auth/me
Authorization: Bearer <accessToken>
```

Пример ответа

```
{
  "user": {
    "id": 1,
    "name": "Alex",
    "email": "alex@example.com",
    "role": "user"
  }
}
```

Замените `<accessToken>` на реальный JWT из login response.

13. Проверьте изменённый token

Измените один символ в JWT и повторите запрос к `/api/auth/me`.

Ожидаемый ответ

```
{
  "error": "Invalid or expired token"
}
```

Ожидаемый HTTP status — 401. Изменение подписанных данных делает signature недействительной.

14. Проверьте expiration

Для быстрой проверки временно установите `JWT_EXPIRES_IN=10s`, перезапустите сервер и выполните login.

1. Сразу выполните `GET /api/auth/me` с новым token.
2. Убедитесь, что запрос успешен.
3. Подождите более 10 секунд.
4. Повторите тот же запрос.
5. Убедитесь, что сервер возвращает 401.

После теста

Верните `JWT_EXPIRES_IN=15m` после проверки expiration.

15. Проверьте неправильный Bearer header

Негативные проверки

Authorization header	Ожидаемый результат
----------------------	---------------------

Authorization header	Ожидаемый результат
отсутствует	401 Authentication required
Token <jwt>	401 Authentication required
Bearer	401 Authentication required
Bearer invalid-token	401 Invalid or expired token
Bearer <валидный JWT>	Запрос выполняется

16. Защите дополнительный endpoint

Добавьте `requireAuth` к одному endpoint, связанному с текущим пользователем, например `GET /api/orders/my`. Пока достаточно убедиться, что route недоступен без JWT.

Пример route

```
router.get(
  "/my",
  requireAuth,
  getMyOrders
);
```

Самостоятельное задание

- Создайте новый User и выполните login.
- Получите access JWT.
- Определите Header, Payload и Signature.
- Найдите в Payload claims `sub`, `iat` и `exp`.
- Выполните protected request без token.
- Выполните protected request с корректным Bearer token.
- Измените token и убедитесь, что signature validation не проходит.
- Проверьте expired token.
- Добавьте `requireAuth` к ещё одному route.
- Убедитесь, что `passwordHash` и `JWT_SECRET` не появляются в JWT payload и HTTP response.

Требования к результату

- Установлен пакет `jsonwebtoken`.
- JWT secret хранится в `.env`.
- Login возвращает `accessToken`.
- Token имеет ограниченный срок действия.
- В JWT используется `sub` для id пользователя.
- JWT передаётся через `Authorization: Bearer`.
- `requireAuth` проверяет JWT через `jwt.verify()`.
- При успешной проверке создаётся `req.user`.
- `GET /api/auth/me` недоступен без token.
- Неверный или expired token возвращает 401.
- `passwordHash` и `JWT_SECRET` отсутствуют в JWT payload.

Критерии оценивания

№	Критерий
1	jsonwebtoken установлен
2	JWT_SECRET находится в environment
3	Секрет не захардкожен в исходном коде
4	createAccessToken() использует jwt.sign()
5	Token содержит sub
6	Token имеет exp
7	Login возвращает accessToken
8	verifyAccessToken() использует jwt.verify()
9	При verify явно ограничен HS256
10	Проверяются issuer и audience
11	requireAuth корректно извлекает Bearer token
12	requireAuth создаёт req.user
13	Protected endpoint без token возвращает 401
14	Изменённый или expired token возвращает 401
15	Студент объясняет различие decode и verify